

Online VAE-Based Acoustic Event Detection on a Microcontroller Edge Device

Walter Zimmer — microPAM V4, 2026

Abstract

We describe an online Variational Autoencoder (VAE) system for acoustic event detection and classification running in real time on a Teensy 4.1 ARM Cortex-M7 microcontroller at 192 kHz sampling rate. The key design principle is that the models learn exclusively from background noise frames and are frozen during detected events, so that the learned latent space represents the ambient acoustic environment rather than any specific signal class. Event detection is provided by a separate SNR tracker; the VAE anomaly score then characterises how far a detected frame departs from the learned background. Four parallel VAEs each specialise on one quarter of the directional intensity spectrum, giving spatially and spectrally resolved representations with a total of 16 exported latent means per processing frame. A Python simulator mirrors the full firmware chain — including a physically motivated ocean ambient noise model — and is used to validate the architecture before deployment. The trained model is persisted to the SD card and restored across power cycles and hibernation intervals, enabling long-term autonomous deployment.

1. Introduction

Battery-operated Passive Acoustic Monitors (PAMs) deployed for weeks to months must detect and log acoustic events without human supervision, with severe constraints on compute, memory, and power. Classical approaches either record raw audio (large storage, no onboard intelligence) or apply fixed-threshold detectors (brittle to changing ambient conditions).

Machine learning on the edge offers a middle path: a model that adapts continuously to the local noise environment and flags departures from it as candidate events. Autoencoders are well suited to this task because they can be trained unsupervised on unlabelled background data and produce an anomaly score — the reconstruction error — that rises when the input no longer resembles the training distribution. Variational Autoencoders additionally structure the latent space to be compact and smooth, which helps distinguish event classes by their position in latent space.

The key constraint distinguishing our setting from offline deep learning is that training and inference must interleave in real time, at the pace of the acoustic acquisition loop, within the interrupt budget of a single microcontroller core.

2. Hardware and Signal Chain

2.1 Platform

The Teensy 4.1 (NXP i.MX RT1062, ARM Cortex-M7 at up to 600 MHz) hosts a four-channel TDM ADC front end (TLV320ADC6140) arranged as a tetrahedral microphone array. Raw audio arrives via DMA at 192 kHz; each DMA half-buffer contains 2048 interleaved 32-bit samples (512 samples per channel). An SD card stores compressed data files and configuration.

2.2 Acoustic Preprocessing

Each DMA completion triggers `process()`, which snapshots the acquisition buffer and pends an asynchronous DSP interrupt (IRQ_QTIMER3, priority below DMA) so the acquisition loop returns immediately. Inside the DSP ISR, the following chain runs:

Spectral analysis. A 50%-overlap RFFT (1024-point Hann-windowed FFT, ARM CMSIS-DSP `arm_rfft_fast_f32`) is applied to each channel independently. The output $Z[k, ch]$ holds complex spectral coefficients for frequency bin k and channel ch .

Directional sound intensity. For each frequency bin k , the imaginary part of the cross-spectrum between every channel pair is computed:

$$\text{Im}(\overline{Z_a} \cdot Z_b) = Z_{a,\text{re}} \cdot Z_{b,\text{im}} - Z_{a,\text{im}} \cdot Z_{b,\text{re}}$$

A fixed mixing matrix $\mathbf{M}$ (3×6) derived from the tetrahedral geometry projects the six pairwise cross-spectra onto three Cartesian intensity components $I_x[k], I_y[k], I_z[k]$. The intensity magnitude spectrum is

$$D[k] = \sqrt{I_x[k]^2 + I_y[k]^2 + I_z[k]^2}$$

D encodes both spectral energy and spatial directionality and serves as the VAE input.

2.3 Background Detection

Before the VAE is invoked, a simple SNR tracker decides whether the current frame is a noise frame or a signal frame:

$$D_{\text{block}} = \frac{1}{N} \sum_{k=0}^{N-1} D[k]$$

$$D_{\text{mean}} += \alpha \cdot (D_{\text{block}} - D_{\text{mean}}) \quad (\text{EMA, noise frames only})$$

$$D_{\text{snr}} = D_{\text{block}} / D_{\text{mean}}$$

with $\alpha = 0.01$ (slow adaptation, memory ≈ 100 frames ≈ 27 s at 192 kHz) and detection threshold $\Theta = 3.0$. The background estimate is updated $10\times$ more slowly on potential signal frames to protect it from contamination.

3. Online VAE Classifier

3.1 Architecture

Four VAEs run in parallel, each receiving one quarter of the 512-bin intensity spectrum D (128 bins per VAE, covering contiguous non-overlapping sub-bands). Each VAE has a symmetric encoder-decoder architecture:

$$\text{input } Q \xrightarrow{\text{ReLU}} H \rightarrow \mu, \log \sigma^2 \xrightarrow{\text{reparam.}} z \xrightarrow{\text{ReLU}} H \rightarrow \hat{D}$$

with $Q = 128$ (input bins per VAE), $H = 32$ (hidden width), $L = 4$ (latent dimensions per VAE), giving a total of $4 \times 4 = 16$ latent means exported per frame.

Layer	Parameters	Size (float32)
W1 [4][H][Q]	encoder input	64 KB (DMAMEM)
W4 [4][Q][H]	decoder output	64 KB (DMAMEM)
Wmu, Wlv [4][L][H]	mu / log-var heads	8 KB
W3 [4][H][L]	decoder hidden	2 KB
b1, b3, b4, bmu, blv	biases	4 KB
Total		≈ 142 KB

The two large weight matrices W1 and W4 are placed in DMAMEM (RAM2) to avoid pressure on tightly coupled RAM1.

3.2 Training Loss

The loss per VAE is the standard ELBO with reconstruction term (MSE) and KL regulariser:

$$\mathcal{L}_v = \underbrace{\frac{1}{Q} \sum_{j=0}^{Q-1} (\hat{D}_j - D_j)^2}_{\text{MSE}} + \beta \underbrace{\frac{1}{2} \sum_{l=0}^{L-1} (e^{\sigma_l^2} + \mu_l^2 - 1 - \sigma_l^2)}_{\text{KL}}$$

with $\beta = 10^{-3}$ and learning rate $\eta = 10^{-4}$ (SGD). Weight initialisation uses Xavier uniform scaling.

3.3 Background-Only Learning

The central design choice is that the backward pass (weight update) runs only on noise frames ($D_{\text{snr}} < \Theta$). On signal frames the forward pass runs for inference but gradients are suppressed. This ensures the VAE latent space represents ambient background conditions; acoustic events produce anomalous encoder outputs precisely because the network has never been trained on them.

In parallel, a per-VAE exponential moving average tracks the background latent position and reconstruction quality:

$$\begin{aligned}\mu_{\text{bg},v} &+= \alpha \cdot (\mu_v - \mu_{\text{bg},v}) \\ \varepsilon_{\text{bg},v} &+= \alpha \cdot (\varepsilon_v - \varepsilon_{\text{bg},v})\end{aligned}$$

where ε_v is the per-VAE MSE reconstruction error and updates are restricted to noise frames.

3.4 Anomaly Score

The per-VAE anomaly score combines latent deviation from the background with elevated reconstruction error:

$$s_v = \|\mu_v - \mu_{\text{bg},v}\| \times \frac{\varepsilon_v}{\varepsilon_{\text{bg},v}}$$

This score is near zero for noise frames (small latent shift, normal error) and rises for events that either move the latent mean (novel spectral structure) or produce high reconstruction error (out-of-distribution input), or both. The four per-VAE scores $s_0 \dots s_3$ each correspond to a distinct spectral sub-band of the intensity spectrum, providing frequency-resolved anomaly information.

3.5 IRQ Structure

The classifier runs in a dedicated interrupt IRQ_QTIMER4 (priority 128, below the DSP ISR) so that the DSP ISR can return before the classifier computation — which takes several hundred microseconds — completes. The DSP ISR triggers the classifier ISR by posting the pending IRQ, passing a snapshot of D and the full intensity matrix I . The classifier ISR then runs the four forward+backward passes and pushes the result vector (header + latent means, and optionally the intensity matrix on signal frames) to the SD write queue.

4. Simulator

A Python simulator (Python/simulator.py) mirrors the complete firmware chain in a single MicroPAM class. All constant names, variable names, weight array dimen-

sions, and arithmetic are kept identical to the C source to make numerical comparison straightforward. Float32 is used throughout.

4.1 Ocean Noise Model

The simulator uses the Wenz (1962) empirical model of ocean ambient noise as a realistic background source. The noise power spectral density (PSD) in dB re 1 $\mu\text{Pa}^2/\text{Hz}$ is the incoherent sum of four components:

Component	Dominant range	Formula
Turbulence	$< 10 \text{ Hz}$	$17 - 30 \log_{10}(f_{\text{kHz}})$
Shipping	$10 \text{ Hz} - 1 \text{ kHz}$	$20 + 12w + 20 \log_{10}(f) - 30 \log_{10}(f^2 + (15 + 4w)/10^4)$
Wind (Knudsen)	$100 \text{ Hz} - 50 \text{ kHz}$	$44 + 28 \log_{10}(1 + v^{0.75}) + 19 \log_{10}(f) - 18 \log_{10}(f^2 + 1/\phi^2)$
Thermal	$> 50 \text{ kHz}$	$-15 + 20 \log_{10}(f_{\text{kHz}})$

where $w \in \{0, 1, 2\}$ is the shipping density, v is wind speed (m/s, converted from Beaufort sea state via $v = 27.6(S/10)^{1.4}$), and $\phi = 2.78 + 0.14v^{0.75}$. The PSD is converted to a spectral amplitude filter applied in the frequency domain; four independent channels are generated (spatially uncorrelated, consistent with the ambient noise assumption).

4.2 Synthetic Signal Injection

To validate event detection and the per-VAE classification, a synthetic multi-step FM chirp signal is injected into the acquisition buffer every 100 processing frames. The signal model is a power-law amplitude envelope with linear FM sweep:

$$s(t) = A \cdot (at)^b \cdot e^{-(at)^c} \cdot \cos(2\pi(f_0 + f_m t)t + \phi_0)$$

with $a = 5000$, $b = 2$, $c = 1.5$, $f_m = 15 \times 10^6 \text{ Hz/s}$, and amplitude $A = 2^{30}$ counts. The injection spans four consecutive DMA buffers (steps 0–3), each with a different starting frequency $f_0 = F_s(n/8 + 1/64)$ for step n , covering four distinct spectral regions. Per-channel time delays $[0, 20, 10, 10]$ samples impose a directional signature that produces non-trivial intensity patterns I_x, I_y, I_z .

The four injection steps are designed to activate different VAE sub-bands preferentially, enabling a simple classifier: the VAE with the highest anomaly score above its noise-estimated threshold ($\mu_{\text{noise}} + 3\sigma_{\text{noise}}$) identifies the event type.

5. Model Persistence

Because the VAE adapts continuously to the local acoustic environment, the learned weights are valuable and should survive power interruptions. On startup, `classifier_init()` calls `classifier_load()`, which reads the full model state — all weight matrices, biases, and background EMA accumulators — from `/VAE_model.dat` on the SD card root. If the file is absent or has an invalid magic header (0x56414531), random Xavier initialisation is used instead.

Before any hibernation event (duty-cycle sleep or hour-window sleep), and at each new recording hour, `classifier_save()` writes the current model to the SD card. If `/VAE_model.dat` already exists it is first renamed to `/VAE_backup/VAE_model_YYYYMMDD_HHMM.dat` (the `/VAE_backup/` subdirectory is created automatically on the first save), preserving a timestamped history of model snapshots organised in a dedicated folder. The total model size is approximately 142 KB.

6. Output Format

In PROC_MODE 4, every processed frame produces a 24-word header in the SD write queue:

Words	Content
[0]	Magic 0x55555555
[1]	<code>millis()</code> firmware timestamp
[2]	<code>signal_flag</code> (1 = event detected, 0 = noise)
[3]	<code>detection_excess</code> = $D_{\text{snr}} - \Theta$ (float)
[4-7]	<code>vae_detect</code> [4] per-VAE anomaly scores (float)
[8-23]	<code>vae_mu</code> [16] latent means, 4 per VAE (float)

On signal frames an additional 3×512 -word intensity matrix I is appended atomically in the same queue block via `push_pair()`, enabling post-hoc spectral and directional analysis of detected events.

7. Discussion

Background-only training as a design principle

The decision to train only on noise frames side-steps the core challenge of supervised edge classifiers: the absence of labelled training data in the field. The VAE learns whatever the ambient environment sounds like during deployment — shipping noise, biological soundscape, flow noise — and treats departures as anomalies. This is particularly appropriate for PAM applications where the target signals (cetacean vocalisations, fish sounds, anthropogenic events) are sparse and diverse.

Spectral sub-band specialisation

Dividing the 512-bin intensity spectrum among four VAEs rather than training a single VAE on the full spectrum serves two purposes. First, it reduces the per-VAE parameter count and computation to fit within the IRQ time budget ($\sim 600 \mu\text{s}$ at 192 kHz, 4 channels, 600 MHz CPU). Second, sub-band specialisation means that events confined to a narrow frequency range (e.g. a tonal call) activate primarily one VAE, providing frequency localisation of the anomaly without explicit signal-processing steps.

Limitations

The anomaly score s_v is a product of two factors that can each fluctuate independently, so its absolute value depends on the stability of the background EMA. Slow environmental transitions (e.g. diel changes in snapping shrimp intensity) may require tuning α to avoid the background tracker following the transition too slowly (false detections) or too quickly (missing the transition signal). The 3σ threshold used in the simulator is post-hoc; a real-time adaptive threshold is a natural extension.

8. Conclusion

We have demonstrated a complete online VAE pipeline for acoustic anomaly detection on a Cortex-M7 microcontroller, operating in real time at 192 kHz with four microphone channels. The system learns continuously from background noise, requires no labelled training data, persists its model across power cycles, and exports compact latent representations and anomaly scores for all processed frames to an SD card. The Python simulator, which mirrors the firmware line-by-line and includes a physically motivated ocean noise model and synthetic signal injection framework, provides a validated development and testing environment for the algorithm before field deployment.

9. Post-Processing Analysis

A companion Python script `Python/vae_model_analysis.py` reads a folder of timestamped model snapshots produced by `classifier_save()` and generates a multi-page PDF report. It parses the binary file layout (magic header + weight matrices in the same order as the firmware writer), extracts background EMA state (`recon_bg`, `mu_bg`), and computes Frobenius norms for `W1` and `W4`.

`Python\.venv\Scripts\python.exe Python\vae_model_analysis.py <path-to-VAE_backup-folder>`

The report contains:

Page	Content
1	Summary text: files analysed, convergence metrics, key findings

Page	Content
2	Background reconstruction error <code>recon_bg</code> on a log scale — shows rapid convergence in the first hour
3	Frobenius norms of <code>W4</code> (solid) and <code>W1</code> (dotted) — <code>W1</code> is effectively frozen; <code>W4</code> adapts gradually
4	Mean <code>mu_bg</code> per VAE over time — reveals slow diel drift of the ambient background
5	<code>mu_bg</code> by latent dimension for VAE 0 — shows which dimensions encode the dominant spectral features

The script is self-contained (numpy + matplotlib only) and can be run on any machine without the firmware build environment.

microPAM V4 — Copyright © 2026 Walter Zimmer. Released under the MIT License.